

Authentication & the OnBase API Server

- [Authentication & the OnBase API Server](#)
 - [Introduction](#)
 - [Choosing an OAuth Grant Type](#)
 - [Setting up the Hyland IdP for Grant Types](#)
 - [Implementing Authentication Flows](#)
 - [Using the Access Token](#)
 - [Other Important Resources](#)

Introduction

To authenticate with the OnBase API Server, you need to use the Hyland Identity Provider, often known as the "Hyland IdP". This allows some powerful use cases that make logging into API-based applications look and work exactly like logging into other Hyland products. The Hyland IdP provides support for the [OAuth 2.0](#) and [OpenID Connect](#) standards. For an integrator or application developer to make the effective use of the Hyland IdP, you'll need to understand your application's architecture.

This document is intended for customers and solution providers writing integrations with OnBase. Internal audiences should read the more specific guidance available from the Identity team.

Choosing an OAuth Grant Type

OAuth 2.0 provides many different grant types, but there are two that we recommend for OnBase use cases:

Authorization Code with PKCE: This is the most common grant type. Use it for any application that a user would log into or would have access to through single sign-on (SSO). This includes all Web and mobile applications and desktop applications, whether the application is built directly to work with OnBase or a third-party application that integrates with it. [PKCE](#) is an acronym of "Proof Key for Code Exchange". It is an extension to the OAuth 2.0 Authentication Code flow that boasts added security ([IETF RFC 7636](#)). Due to the added security the Auth Code flow with PKCE is now always preferred over the [Implicit authentication flow](#) for client applications.

Resource Owner Password: This grant type is used **only** for trusted services with no UI that always work in the context of a given user. An example of such a service would be an application that automatically polls an external system to make updates to OnBase. This grant type does not support SSO or other advanced authentication features. This limitation is OK, since it should only be used in backend scenarios where the credentials are protected in a trusted environment. (If you research OAuth flows online, you'll see that this grant type is deprecated in most cases. Indeed, it lacks security and robust features for any scenario that requires a user to log in, which is why we recommend the Auth Code with PKCE grant for all such cases.)

Setting up the Hyland IdP for Grant Types

Depending on which grant types have been chosen there may be additional identity provider configuration necessary to support those grant types. For an up to date guide for grant type configurations please refer to the latest version of the Identity and Access Management Services Manual Reference Guide (MRG), which can be found on the Hyland Community's [Authentication and Security](#) page.

Implementing Authentication Flows

Because OAuth 2.0 is an open and common standard, it's possible to research and implement these flows yourself. However, we recommend you take advantage of one of the many third-party libraries available, which will save you from having to do most of

the boilerplate work and avoid common mistakes. Libraries are available for many platforms—Web, native, mobile—and different licenses, either open source or proprietary. OpenID Connect helpfully offers a [certified list of these libraries](#), organized by development languages.

Using the Access Token

If the authentication flow has been executed correctly then the Hyland IdP will generate and return an access token to the client. An access token is simply a string that allows applications to make requests on behalf of a user. The token itself may not have credentials or identifying information in it but the Hyland IdP will recognize it and be capable of validating a request based on the token. When the client then makes future requests directly to the OnBase API Server it should also include the access token. The OnBase API Server will then make a request to the Hyland IdP to validate the access token. If the Hyland IdP validates the token successfully then the OnBase API Server fulfills the request and passes the response to the client.

The Hyland IdP issues and expects a type of access token called a [Bearer token](#). When making requests it is necessary for the client to identify the token as a Bearer token by prepending the string "Bearer " to the access token. For example if the access token returned by the Hyland IdP was of the form "1234" then the client would modify the token to a new value of "Bearer 1234". All requests to the OnBase API Server should then include an "Authorization" header containing the modified token. If the credentials supplied to the Hyland IdP were correct and the Bearer token was successfully validated then the OnBase API Server will execute and return the response to the client.

Other Important Resources

For additional details on session creation, heartbeat, and disconnect behavior, refer to the [Session Lifecycle](#) guide.